

Guía Metodológica Detallada

Optimización de Carteras con Machine Learning, Regímenes de Mercado y Métodos Cuantitativos

Jaime

Documento de trabajo interno (no entregable)

Índice

1. Objetivo general del proyecto	4
2. Entorno tecnológico y librerías	4
3. Mercado, universo de activos y horizonte temporal	5
3.1. Mercado y benchmark	5
3.2. Universo de activos	5
3.3. Horizonte temporal	5
4. Datos: descarga, limpieza y estructura	5
4.1. Descarga de datos	5
4.2. Limpieza y alineación	6
4.3. Estructura de datos	6
5. Construcción del target mensual	6
5.1. Definición del target	6
5.2. Implementación	7
6. Construcción del dataset mensual	7
6.1. Ventanas temporales	7
6.2. Estructura final del dataset	7
7. Features finales para el modelo mensual	7
7.1. Retornos pasados	8
7.2. Volatilidad	8

7.3. Tendencia (medias móviles)	8
7.4. Features basadas en volumen	8
7.5. Features basadas en OHLC (derivadas, no en bruto)	8
7.6. Indicadores técnicos opcionales	9
8. Pipeline general y esquema rolling window	9
9. Modelos predictivos	10
9.1. Modelos shallow (ML clásico)	10
9.2. Modelos deep (PyTorch)	10
9.2.1. LSTM pequeña	10
9.2.2. LSTM mediana	10
9.2.3. CNN simple	11
9.2.4. CNN mediana	11
9.3. Entrenamiento de modelos naive	11
10. Normalización y preparación de datos	11
11. Detección de Regímenes de Mercado mediante Clustering con Distancia Wasserstein	12
11.1. Motivación	12
11.2. Construcción de distribuciones de retornos	12
11.3. Distancia Wasserstein entre ventanas	13
11.4. Clustering no supervisado	13
11.5. Interpretación económica de los regímenes	13
11.6. Asignación de regímenes en el tiempo	14
11.7. Integración en el pipeline	14
12. Selección de modelo por régimen	14
12.1. Método principal: selección del mejor modelo naive por régimen	15
12.2. Extensión opcional: reentrenar modelos específicos por régimen	16
13. Optimización de carteras	16
13.1. Planteamiento general	16
13.2. Implementación con <code>riskfolio-lib</code>	16
14. Simulación Monte Carlo Robusta	17
14.1. Motivación y fundamento teórico	17
14.2. Modelo probabilístico para los retornos futuros	17

14.3. Generación de escenarios	18
14.4. Optimización por escenario	18
14.5. Agregación robusta de pesos	19
14.5.1. Media de pesos	19
14.5.2. Mediana de pesos	19
14.5.3. Percentil conservador	19
14.6. Interpretación financiera	19
14.7. Relación con la teoría de decisión bajo incertidumbre	20
14.8. Conclusión	20
15. Costes de transacción	20
16. Evaluación	21
16.1. Evaluación estadística básica	21
16.2. Evaluación estadística avanzada: tests de significancia	21
16.2.1. Tests para regresión lineal (Statsmodels)	21
16.2.2. Tests para comparar modelos	21
16.2.3. Tests sobre la utilidad predictiva	22
16.3. Evaluación financiera	22
17. Extensión opcional: modelado intradía con Transformers	22
17.1. Datos intradía	22
17.2. Construcción de secuencias	22
17.3. Tarea de predicción	22
17.4. Modelos	23
17.5. Métricas	23
18. Cierre del diseño metodológico	23

1. Objetivo general del proyecto

El objetivo de este proyecto es diseñar y documentar un pipeline cuantitativo completo para:

- Predecir retornos mensuales de un conjunto de acciones del S&P 500 utilizando modelos de Machine Learning (ML) y Deep Learning (DL).
- Detectar regímenes de mercado y adaptar la elección de modelo a cada régimen.
- Optimizar carteras de forma robusta, incorporando incertidumbre mediante simulación Monte Carlo y costes de transacción.
- Evaluar el rendimiento de las carteras resultantes frente a benchmarks clásicos.

Este documento es una guía técnica cerrada: define qué se va a hacer, cómo se va a hacer, con qué librerías y con qué diseño final. No es la memoria del TFG, sino la base metodológica sobre la que se construirá.

2. Entorno tecnológico y librerías

El proyecto se implementará en Python. El entorno base será:

- **Versión de Python:** 3.10+ (idealmente 3.10 o 3.11).
- **Gestor de entorno:** conda o venv.

Librerías principales:

- **Manipulación de datos:** pandas, numpy.
- **Descarga de datos:** yfinance.
- **Machine Learning clásico:** scikit-learn.
- **Gradient boosting:** xgboost.
- **Deep Learning:** pytorch (y torchvision si fuera necesario).
- **Optimización de carteras:** riskfolio-lib.
- **Métricas financieras:** empyrical.
- **Cálculo de distancia Wasserstein:** scipy.stats.

- **Indicadores técnicos (opcional):** `ta` o implementación manual.

El código se organizará en módulos (por ejemplo, `data_processing.py`, `features.py`, `models.py`, `regimes.py`, `optimization.py`, `backtest.py`) para mantener claridad y reutilización.

3. Mercado, universo de activos y horizonte temporal

3.1. Mercado y benchmark

- **Mercado:** Estados Unidos.
- **Benchmark principal:** S&P 500 (índice).

3.2. Universo de activos

- **Universo:** 30 acciones de gran capitalización pertenecientes al S&P 500.
- **Criterios de selección:**
 - Alta liquidez.
 - Diversidad sectorial (tecnología, financiero, consumo, salud, energía, industria, etc.).
 - Historial suficiente en el periodo de estudio.

3.3. Horizonte temporal

- **Periodo bruto de datos:** 2010–2026 (diario).
- **Periodo efectivo de backtesting:** 2013–2026.

Los primeros años se utilizan para construir ventanas históricas suficientes para features y regímenes.

4. Datos: descarga, limpieza y estructura

4.1. Descarga de datos

Se utilizará `yfinance` para descargar datos diarios:

- Precio de cierre ajustado (`Adj Close`).

- Volumen.
- OHLC (Open, High, Low, Close) para la extensión intradía y posibles features derivados.

4.2. Limpieza y alineación

- Alinear todos los activos en un mismo índice temporal.
- Rellenar festivos y días sin cotización con NaN y evitar forward-fill en precios.
- Eliminar activos con huecos excesivos en el periodo de estudio.

4.3. Estructura de datos

Se trabajará con:

- **DataFrame de precios diarios:** filas = fechas, columnas = activos.
- **DataFrame de volumen diario:** misma estructura.
- **DataFrame de retornos diarios:** calculados como log-retornos.

5. Construcción del target mensual

El objetivo es predecir el log-retorno mensual del mes siguiente para cada activo.

5.1. Definición del target

Sea $P_t^{(i)}$ el precio de cierre ajustado del activo i en el día t . Definimos el log-retorno mensual del activo i entre el inicio del mes m y el final del mes m como:

$$r_m^{(i)} = \ln \left(\frac{P_{\text{fin mes } m}^{(i)}}{P_{\text{inicio mes } m}^{(i)}} \right)$$

El target a predecir en el último día del mes m será:

$$y_{m+1}^{(i)} = r_{m+1}^{(i)}$$

5.2. Implementación

- Usar `resample('M')` para identificar el último día de cada mes.
- Para cada activo, obtener el precio del primer y último día de cada mes.
- Calcular el log-retorno mensual.
- Desplazar el target un mes hacia adelante para asociarlo a las features del mes anterior.

6. Construcción del dataset mensual

Aunque los datos son diarios, el modelo trabaja a nivel mensual. Para cada mes m y activo i :

- Se calculan features usando datos hasta el último día del mes m .
- Se asocia el target $y_{m+1}^{(i)}$.

6.1. Ventanas temporales

Se utilizarán ventanas móviles sobre datos diarios para construir features:

- 1 mes $\approx$ 21 días.
- 3 meses $\approx$ 63 días.
- 6 meses $\approx$ 126 días.

6.2. Estructura final del dataset

Cada fila del dataset mensual corresponde a un par (activo i , mes m) y contiene:

- Features calculadas con datos hasta fin de mes m .
- Target $y_{m+1}^{(i)}$.

7. Features finales para el modelo mensual

El diseño de features busca un equilibrio entre:

- Información suficiente para capturar patrones.
- Simplicidad y robustez (evitar overfitting).
- Interpretabilidad razonable.

7.1. Retornos pasados

- Log-retorno 1 mes:

$$r_{1m}^{(i)}(m) = \ln \left(\frac{P_{\text{fin mes } m}^{(i)}}{P_{\text{fin mes } m-1}^{(i)}} \right)$$

- Log-retorno acumulado 3 meses.
- Log-retorno acumulado 6 meses.

7.2. Volatilidad

- Volatilidad 1 mes: desviación estándar de los log-retornos diarios de los últimos 21 días.
- Volatilidad 3 meses: desviación estándar de los últimos 63 días.

7.3. Tendencia (medias móviles)

- MA20: media móvil de 20 días del precio de cierre.
- MA60: media móvil de 60 días.
- Ratio MA20/MA60: indicador de tendencia relativa.

7.4. Features basadas en volumen

- Volumen medio 20 días.
- Volumen medio 60 días.
- Ratio volumen actual / volumen medio 20 días.

7.5. Features basadas en OHLC (derivadas, no en bruto)

Aunque el pipeline mensual no necesita OHLC en bruto, sí es útil derivar algunas features:

- Rango intradía medio del mes:

$$\text{Rango}_{\text{med}}(m) = \text{media}_{t \in \text{mes } m}(\text{High}_t - \text{Low}_t)$$

- Cuerpo medio de la vela:

$$\text{Cuerpo}_{\text{med}}(m) = \text{media}_{t \in \text{mes } m} |\text{Close}_t - \text{Open}_t|$$

- True Range medio (para capturar gaps).

Estas features se calculan con `pandas` y, si se desea, con ayuda de librerías como `ta`.

7.6. Indicadores técnicos opcionales

Opcionales, pero útiles si el tiempo lo permite:

- RSI 14 días.
- ATR (Average True Range) 14 días.

8. Pipeline general y esquema rolling window

El pipeline se ejecuta en un esquema *rolling* mensual. Para cada mes m en el periodo de backtesting:

1. Construir el dataset con datos hasta fin de mes m (features y targets históricos).
2. Ajustar el escalado (`StandardScaler`) con datos hasta m .
3. Entrenar modelos “naive” (sin regímenes) con datos hasta m .
4. Evaluar y guardar resultados de los modelos naive.
5. Detectar el régimen de mercado en el mes m .
6. Seleccionar el modelo adecuado según el régimen (método principal).
7. Generar predicciones de retornos para el mes $m + 1$.
8. Generar escenarios Monte Carlo.
9. Optimizar la cartera para el mes $m + 1$.
10. Aplicar costes de transacción.
11. Registrar el retorno neto del mes $m + 1$.

Este proceso se repite mes a mes, sin utilizar información futura (no hay *data leakage*).

9. Modelos predictivos

9.1. Modelos shallow (ML clásico)

Se utilizarán los siguientes modelos de `scikit-learn` y `xgboost`:

- Regresión lineal (`LinearRegression`).
- Ridge, Lasso, Elastic Net.
- Random Forest (`RandomForestRegressor`).
- XGBoost (`XGBRegressor`).
- MLP simple (`MLPRegressor`, opcional).

Los hiperparámetros se fijarán de forma razonable (no se hará una búsqueda extrema para evitar sobreajuste y consumo excesivo de tiempo).

9.2. Modelos deep (PyTorch)

Se utilizarán modelos sencillos pero representativos:

9.2.1. LSTM pequeña

- Capa LSTM con 32 unidades.
- Dropout 0.2.
- Capa densa final con 1 neurona (salida escalar).

9.2.2. LSTM mediana

- Capa LSTM con 64 unidades.
- Dropout 0.3.
- Capa densa intermedia de 32 neuronas.
- Capa densa final de 1 neurona.

9.2.3. CNN simple

- Conv1D(32 filtros, kernel=3).
- MaxPool1D.
- Flatten.
- Dense(32).
- Dense(1).

9.2.4. CNN mediana

- Conv1D(32, kernel=3).
- Conv1D(16, kernel=3).
- MaxPool1D.
- Flatten.
- Dense(32).
- Dense(1).

9.3. Entrenamiento de modelos naive

En la primera fase, todos los modelos se entrenan sin distinguir regímenes de mercado. Esto proporciona:

- Un baseline claro.
- Resultados comparables entre modelos.
- Datos para analizar qué modelos funcionan mejor en qué entornos.

10. Normalización y preparación de datos

Se utilizará `StandardScaler` de `scikit-learn`:

- Ajustado únicamente con datos disponibles hasta el mes m .
- Aplicado a todas las features numéricas.

Para modelos deep, se utilizarán tensores de PyTorch, manteniendo el mismo escalado.

11. Detección de Regímenes de Mercado mediante Clustering con Distancia Wasserstein

La detección de regímenes de mercado es un componente central del pipeline, ya que permite adaptar la elección del modelo predictivo al entorno estadístico en el que se encuentra el mercado. En este proyecto, la detección de regímenes no se basa en reglas predefinidas ni en indicadores técnicos, sino en un enfoque no supervisado que identifica regímenes directamente a partir de la estructura estadística de los retornos. Para ello, se emplea un método de clustering basado en la distancia Wasserstein entre distribuciones de retornos, siguiendo la metodología propuesta en la literatura reciente de análisis de regímenes.

11.1. Motivación

Los mercados financieros presentan dinámicas que cambian en el tiempo: periodos de estabilidad, fases de alta volatilidad, episodios de caídas pronunciadas, etc. Estos cambios no siempre son evidentes a partir de indicadores simples, y pueden reflejarse mejor en la forma completa de la distribución de retornos que en métricas puntuales como la media o la volatilidad.

El uso de la distancia Wasserstein permite comparar distribuciones de retornos completas, capturando diferencias en:

- media,
- volatilidad,
- asimetría (skewness),
- curtosis (colas pesadas),
- forma general de la distribución.

Esto permite detectar regímenes de mercado de manera más rica y robusta que con métodos basados únicamente en indicadores técnicos.

11.2. Construcción de distribuciones de retornos

El procedimiento comienza dividiendo el histórico del índice de mercado (S&P 500 o SPY) en ventanas móviles de longitud fija, típicamente 60 días. Para cada ventana w , se obtiene la distribución empírica de retornos diarios:

$$\mathcal{D}_w = \{r_t, r_{t+1}, \dots, r_{t+59}\}$$

Cada ventana representa un estado local del mercado.

11.3. Distancia Wasserstein entre ventanas

Para cada par de ventanas (i, j) se calcula la distancia Wasserstein-2 entre sus distribuciones empíricas:

$$d_{ij} = W_2(\mathcal{D}_i, \mathcal{D}_j)$$

La distancia Wasserstein mide el “coste mínimo” de transformar una distribución en otra, capturando diferencias en toda la forma de la distribución, no solo en momentos concretos.

Esto produce una matriz de distancias:

$$D = [d_{ij}] \in \mathbb{R}^{T \times T}$$

donde T es el número de ventanas.

11.4. Clustering no supervisado

A partir de la matriz de distancias D , se aplica un método de clustering no supervisado para identificar grupos de ventanas con distribuciones similares. Se emplea clustering jerárquico o k-means sobre un embedding obtenido mediante técnicas como MDS (Multidimensional Scaling).

El número de clusters se fija en $k = 3$, siguiendo la literatura y la interpretación financiera estándar de tres regímenes principales.

El resultado es un conjunto de clusters:

$$C_1, C_2, C_3$$

Cada cluster representa un régimen estadístico distinto del mercado.

11.5. Interpretación económica de los regímenes

Aunque los regímenes se obtienen de manera no supervisada, es posible interpretarlos económicamente analizando las características medias de las ventanas pertenecientes a cada cluster:

- retorno medio,
- volatilidad media,

- skewness,
- curtosis.

Esto permite mapear los clusters a regímenes financieros clásicos:

- **Régimen 1 (Bull Market):** distribuciones estrechas, volatilidad baja o moderada, retornos positivos.
- **Régimen 2 (Bear Market):** distribuciones anchas, alta volatilidad, colas pesadas, retornos negativos.
- **Régimen 3 (Sideways / Mean-Reverting):** retornos cercanos a cero, volatilidad intermedia, forma de distribución estable.

Este mapeo permite mantener interpretabilidad financiera sin renunciar a la potencia del enfoque no supervisado.

11.6. Asignación de regímenes en el tiempo

Cada ventana w recibe una etiqueta de régimen según el cluster al que pertenece. La etiqueta del mes m se asigna según la ventana que termina en dicho mes.

Esto produce una serie temporal de regímenes:

$$R_m \in \{1, 2, 3\}$$

que se utiliza posteriormente para seleccionar el modelo predictivo más adecuado para cada entorno.

11.7. Integración en el pipeline

El régimen detectado en el mes m determina qué modelo se utilizará para predecir los retornos del mes $m + 1$. Para cada régimen, se selecciona el modelo que históricamente ha mostrado mejor desempeño dentro de ese régimen, según las métricas de evaluación definidas en el proyecto.

Este enfoque permite adaptar dinámicamente el modelo predictivo a las condiciones del mercado, mejorando la robustez y la estabilidad del pipeline.

12. Selección de modelo por régimen

Una vez definidos los regímenes, se utilizan para adaptar la elección de modelo.

12.1. Método principal: selección del mejor modelo naive por régimen

Idea

No se reentrenan modelos por régimen. En su lugar:

- Se entrenan todos los modelos naive con todos los datos (hasta cada punto temporal).
- Se analiza, a posteriori, qué modelo funciona mejor en cada régimen.
- En tiempo real, se detecta el régimen y se elige el modelo que históricamente ha funcionado mejor en ese régimen.

Procedimiento

1. Para cada mes histórico m :
 - Se conoce el régimen R_m .
 - Se conocen las predicciones de cada modelo y los retornos reales.
2. Para cada régimen k y modelo j :
 - Filtrar los meses con $R_m = k$.
 - Calcular métricas:
 - MAE, RMSE.
 - Correlación predicción–real.
 - % acierto de signo.
 - Opcional: Sharpe de la estrategia basada en ese modelo.
3. Para cada régimen k , seleccionar el modelo j^* con mejor desempeño.
4. En el backtest rolling:
 - En el mes m , detectar el régimen R_m .
 - Usar el modelo $j^*(R_m)$ para predecir retornos del mes $m + 1$.

12.2. Extensión opcional: reentrenar modelos específicos por régimen

Como extensión avanzada:

- Para cada régimen k , se entrena un modelo específico usando solo datos de ese régimen.
- En tiempo real, se detecta el régimen y se usa el modelo entrenado para ese régimen.

Riesgos:

- Pocos datos por régimen.
- Mayor riesgo de overfitting.
- Mayor coste computacional.

Por ello, se mantiene como extensión opcional, no como núcleo.

13. Optimización de carteras

13.1. Planteamiento general

Dado un vector de retornos esperados $\hat{r}$ (predicciones) y una matriz de covarianzas Σ , se busca un vector de pesos w que maximice el retorno esperado sujeto a restricciones.

$$\max_w w^\top \hat{r}$$

Sujeto a:

- $w_i \geq 0$ (no cortos).
- $\sum_i w_i = 1$ (total invertido).
- $w_i \leq w_{\text{máx}}$ (por ejemplo, 0.10–0.15).

13.2. Implementación con riskfolio-lib

- Construir un objeto de portafolio con retornos históricos y/o predichos.
- Especificar el modelo de riesgo (covarianza clásica o Ledoit–Wolf).
- Definir restricciones de pesos.

- Resolver el problema de optimización.

Se puede utilizar tanto un enfoque de maximización de retorno para un nivel de riesgo dado como variantes de media-varianza.

14. Simulación Monte Carlo Robusta

La simulación Monte Carlo constituye un componente esencial del pipeline, ya que permite incorporar explícitamente la incertidumbre inherente tanto a las predicciones de retornos como a la estimación de la matriz de covarianzas. En lugar de optimizar la cartera únicamente sobre un único vector de retornos esperados $\hat{r}$, se generan múltiples escenarios plausibles de retornos futuros y se optimiza la cartera en cada uno de ellos. Este enfoque produce carteras más estables, diversificadas y menos sensibles al error de estimación, mitigando uno de los problemas clásicos de la optimización media-varianza.

14.1. Motivación y fundamento teórico

La optimización clásica de Markowitz utiliza un único par $(\hat{r}, \Sigma)$, donde:

$$\hat{r} \in \mathbb{R}^N \quad \text{y} \quad \Sigma \in \mathbb{R}^{N \times N}$$

representan, respectivamente, los retornos esperados y la matriz de covarianzas estimados. Sin embargo:

- $\hat{r}$ es una predicción sujeta a error.
- Σ es una estimación ruidosa basada en una muestra finita.
- La optimización clásica es extremadamente sensible a pequeñas perturbaciones en $\hat{r}$.

Por tanto, optimizar sobre un único punto estimado puede conducir a soluciones inestables y poco realistas. Monte Carlo permite aproximar la distribución de posibles futuros y optimizar sobre ella.

14.2. Modelo probabilístico para los retornos futuros

Se asume que los retornos futuros siguen una distribución normal multivariante:

$$r \sim \mathcal{N}(\hat{r}, \Sigma)$$

Esta elección se justifica por:

- su uso estándar en finanzas cuantitativas,
- su capacidad para capturar correlaciones entre activos,
- su eficiencia computacional,
- su compatibilidad con la optimización convexa.

Aunque la distribución real de retornos no es normal, esta aproximación es suficiente para generar escenarios plausibles y coherentes con la estructura de correlaciones.

14.3. Generación de escenarios

Sea S el número de escenarios Monte Carlo (típicamente $S = 500$ o $S = 1000$). Cada escenario se genera como:

$$r^{(s)} = \hat{r} + Lz^{(s)}, \quad z^{(s)} \sim \mathcal{N}(0, I)$$

donde L es la descomposición de Cholesky de Σ :

$$\Sigma = LL^\top$$

Este procedimiento garantiza que:

- los escenarios respetan las correlaciones entre activos,
- la dispersión de los retornos es coherente con la volatilidad estimada,
- se explora un abanico realista de posibles futuros.

14.4. Optimización por escenario

Para cada escenario $r^{(s)}$ se resuelve el problema:

$$\max_{w^{(s)}} (w^{(s)})^\top r^{(s)}$$

sujeto a:

$$w_i^{(s)} \geq 0, \quad \sum_i w_i^{(s)} = 1, \quad w_i^{(s)} \leq w_{\max}$$

donde $w_{\max}$ es típicamente 0,10–0,15.

Este proceso produce un conjunto de S vectores de pesos:

$$\mathcal{W} = \{w^{(1)}, w^{(2)}, \dots, w^{(S)}\}$$

Cada $w^{(s)}$ representa la cartera óptima bajo un posible futuro coherente con la información disponible.

14.5. Agregación robusta de pesos

El objetivo es obtener un único vector de pesos robusto w^* que sintetice la información contenida en los S escenarios.

14.5.1. Media de pesos

$$w^* = \frac{1}{S} \sum_{s=1}^S w^{(s)}$$

Produce carteras suaves y diversificadas. Es adecuada cuando no se desea un sesgo conservador.

14.5.2. Mediana de pesos

$$w_i^* = \text{mediana} \left(w_i^{(1)}, \dots, w_i^{(S)} \right)$$

Más robusta frente a escenarios extremos o soluciones degeneradas.

14.5.3. Percentil conservador

Para un nivel α (por ejemplo, 25 %):

$$w_i^* = \text{percentil}_\alpha \left(w_i^{(1)}, \dots, w_i^{(S)} \right)$$

Favorece carteras prudentes, reduciendo exposición a activos con alta incertidumbre.

14.6. Interpretación financiera

El enfoque Monte Carlo permite:

- incorporar explícitamente la incertidumbre en las predicciones,
- evitar soluciones extremas típicas de la optimización clásica,
- obtener carteras más estables en el tiempo,

- capturar la dispersión de posibles futuros,
- reducir la sensibilidad a errores en $\hat{r}$ y Σ ,
- mejorar la robustez frente a cambios de régimen.

En esencia, se sustituye la optimización sobre un único futuro estimado por una optimización sobre un abanico de futuros plausibles, lo que produce decisiones más realistas y consistentes con la naturaleza incierta de los mercados financieros.

14.7. Relación con la teoría de decisión bajo incertidumbre

El método puede interpretarse como:

- una aproximación numérica a la optimización estocástica,
- un caso particular de *robust optimization*,
- una forma de suavizar la función objetivo mediante integración sobre la distribución de retornos,
- una técnica para aproximar el valor esperado de la política óptima bajo incertidumbre.

En términos formales, el objetivo implícito es:

$$w^* = \arg \max_w \mathbb{E}_{r \sim \mathcal{N}(\hat{r}, \Sigma)}[w^\top r]$$

pero sujeto a restricciones adicionales de estabilidad y robustez mediante agregación.

14.8. Conclusión

La simulación Monte Carlo robusta es un componente fundamental del pipeline, ya que permite obtener carteras más estables, diversificadas y resistentes al error de estimación. Su integración con la optimización clásica proporciona un equilibrio adecuado entre sofisticación matemática, interpretabilidad y viabilidad computacional.

15. Costes de transacción

Se modelan como proporcionales al volumen de rebalanceo:

$$\text{Coste}_m = c \cdot \sum_i |w_{m,i} - w_{m-1,i}|$$

donde c es el coste por unidad de cambio de peso.

El retorno neto del mes m es:

$$R_{\text{neto},m} = R_{\text{bruto},m} - \text{Coste}_m$$

16. Evaluación

16.1. Evaluación estadística básica

MAE, RMSE, correlación, acierto de signo.

16.2. Evaluación estadística avanzada: tests de significancia

Además de las métricas básicas, se aplicarán tests estadísticos para evaluar rigurosamente la calidad de los modelos y la significancia de sus parámetros.

16.2.1. Tests para regresión lineal (Statsmodels)

Usando `statsmodels.api.OLS` se obtienen:

- **t-tests** para coeficientes individuales:

$$t_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)}$$

Permiten evaluar si cada feature tiene un efecto significativo sobre el retorno.

- **p-values** asociados a cada coeficiente.
- **F-test** de significancia global del modelo.
- **Durbin–Watson** para autocorrelación de residuos.
- **Breusch–Pagan** para heterocedasticidad.
- **Jarque–Bera** para normalidad de residuos.

16.2.2. Tests para comparar modelos

- **Diebold–Mariano test** para comparar errores de predicción entre dos modelos.
- **Test de diferencia de medias** para MAE o RMSE.
- **Test binomial** para evaluar si el acierto de signo es mayor que el 50 %.

16.2.3. Tests sobre la utilidad predictiva

- Test de media de errores = 0.
- Test de correlación predicción–real $\neq 0$.

Estos tests permiten evaluar si las predicciones contienen información estadísticamente significativa.

16.3. Evaluación financiera

Sharpe, volatilidad, retorno anualizado, drawdown, turnover.

Benchmarks: 1/N, Buy & Hold, Markowitz sin ML.

17. Extensión opcional: modelado intradía con Transformers

Esta sección es una extensión independiente del pipeline principal. No afecta al diseño mensual, pero sirve como exploración de arquitecturas modernas.

17.1. Datos intradía

- Activo: SPY (ETF del S&P 500).
- Frecuencia: 5 minutos.
- Periodo: 2020–2024.
- Variables: OHLCV (Open, High, Low, Close, Volume).

17.2. Construcción de secuencias

- Ventanas deslizantes de longitud 100 (500 minutos $\approx$ 1–2 días de trading).
- Cada ventana se representa como una secuencia de vectores OHLCV normalizados.

17.3. Tarea de predicción

Ejemplos posibles:

- Clasificación binaria: subida/bajada en el siguiente intervalo.
- Clasificación multi-clase: rango de retorno.

17.4. Modelos

- LSTM pequeña (baseline).
- Transformer 1D pequeño:
 - Embedding lineal de la entrada.
 - 2–3 capas de atención multi-cabeza.
 - Capa densa final.

17.5. Métricas

- Accuracy.
- F1-score.
- AUC-ROC.

El objetivo de esta extensión es exploratorio: comparar el rendimiento de Transformers frente a LSTM en un entorno intradía.

18. Cierre del diseño metodológico

El diseño final del proyecto queda cerrado con los siguientes elementos clave:

- Predicción mensual de retornos con modelos ML y DL (naive primero).
- Detección de regímenes de mercado con métodos simples (y Wasserstein como extensión).
- Selección dinámica de modelos por régimen (método principal: elegir el mejor modelo naive por régimen).
- Optimización de carteras con `riskfolio-lib`.
- Robustez mediante Monte Carlo.
- Costes de transacción explícitos.
- Evaluación estadística y financiera completa.
- Extensión opcional intradía con Transformers y OHLCV.

Este documento define de forma concreta y cerrada qué se va a hacer y cómo se va a hacer, sirviendo como guía maestra para la implementación y, posteriormente, para la redacción del TFG.